

Explainable Graph Ensemble Learning for Multivariate Time Series Anomaly Detection in Cloud Microservice Architectures

Kevin O'Shea^{1b}, Sen Yan^{1b}, *Graduate Student Member, IEEE*, Ming Yu, Xianjuan Chen, Stefano Mauceri^{1b}, Bhaskar Dhariyal^{1b}, Lei Xu, Noel O'Connor^{1b}, *Member, IEEE*, and Mingming Liu^{1b}, *Senior Member, IEEE*

Abstract—The increasing adoption of cloud microservice architectures demands robust multivariate time series anomaly detection algorithms to ensure a reliable computing infrastructure. Although countless of these algorithms are readily available in the literature, significant concerns remain about their interpretability, particularly in cases where it is required to consider both the temporal and topological aspects of the explanation. Our proposed framework leverages attention-based spatial-temporal graph convolutional networks and ensemble learning to capture complex spatial-temporal dependencies across time series. Our approach achieves an averaged event-wise F1 score of 0.94 on two proprietary datasets. After the detection phase, an analysis of the network attention weights, at carefully selected layers, provides useful explanations of the anomalous events to cloud operators.

Index Terms—Anomaly detection, microservice architecture, deep learning, graph neural networks, explainability.

NOMENCLATURE

List of Abbreviations & Acronyms

AE	Autoencoder.
ALSTM	Attention-Based Long Short-Term Memory.
ASTGCN	Attention-Based Spatial-Temporal Graph Convolutional Network.
BiLSTM	Bidirectional Long Short-Term Memory.
DAM	Dynamic Attention Model.
DL	Deep Learning.
GAT	Graph Attention Network.
GCN	Graph Convolutional Network.

GNN	Graph Neural Network.
GPD	Generalised Pareto Distribution.
GT	Ground Truth.
HITS	Hyperlink-Induced Topic Search.
LIME	Local Interpretable Model-agnostic Explanations.
LSTM	Long Short-Term Memory.
ML	Machine Learning.
MLP	Multi-Layer Perceptron.
MSE	Mean Squared Error.
MTSAD	Multivariate Time Series Anomaly Detection.
PatchTST	Patch Time Series Transformer.
POT	Peaks Over Threshold.
RF	Random Forest.
SHAP	Shapley Additive Explanation.
SOFTS	Series-Core Fused Time Series Forecaster.
STAD	Star Aggregate Dispatch.
STGCN	Spatio-Temporal Graph Convolutional Network.
VAE	Variational Autoencoder.
XAI	Explainable Artificial Intelligence.
xLSTM	Extended Long Short-Term Memory.

I. INTRODUCTION

CLOUD microservice architectures are rapidly replacing conventional monolithic systems due to their scalability, flexibility and maintainability [1]. Recent research has been conducted in areas involving fault isolation, performance prediction, and anomaly detection [2], [3], [4], [5]. Among these, effective monitoring and timely anomaly detection are the most important tasks to ensure reliable, uninterrupted cloud services [6]. Several traditional algorithms have been applied to multivariate time series anomaly detection in cloud systems, such as rule-based approaches [7], statistical process control [8], and principal component analysis-based methods [9]. These techniques usually detect anomalies by first analysing individual (univariate) time series using statistical thresholds, and then aggregating the results to infer multivariate anomalies [10], [11]. While such methods can be effective in well-defined environments, they often struggle to capture the complex inter-variable dependencies of microservice systems, frequently resulting in alert storms [12].

To address these challenges, Machine Learning (ML) approaches, particularly Deep Learning (DL) models, have been employed to capture complex dependencies and adapt to changing service behaviours, and have been shown to achieve better performance than threshold-based methods [13], [14], [15].

Received 18 November 2024; revised 28 July 2025; accepted 14 November 2025. Date of publication 19 November 2025; date of current version 10 March 2026. This work was supported by the Huawei Ireland Research Centre for the multivariate time series anomaly detection project and Taighde Éireann — Research Ireland under Grant SFI/12/RC/2289_P2, in part by the European Regional Development Fund in collaboration with the Research Ireland Insight Centre for Data Analytics at Dublin City University. Recommended for acceptance by K. Djemame. (Kevin O'Shea, Sen Yan, and Ming Yu contributed equally to this work.) (Corresponding author: Mingming Liu.)

Kevin O'Shea, Ming Yu, and Xianjuan Chen are with the Insight Research Ireland Centre for Data Analytics, Dublin City University, Dublin 9, Ireland (e-mail: kevin.oshea@dcu.ie; ming.yu@insight-centre.org; xianjuan.chen@insight-centre.org).

Sen Yan, Noel O'Connor, and Mingming Liu are with the School of Electronic Engineering and Insight Research Ireland Centre for Data Analytics, Dublin City University, Dublin 9, Ireland (e-mail: sen.yan@dcu.ie; noel.oconnor@dcu.ie; mingming.liu@dcu.ie).

Stefano Mauceri, Bhaskar Dhariyal, and Lei Xu are with Huawei Ireland Research Centre, Dublin 2, Ireland (e-mail: stefano.mauceri@huawei.com; bhaskar.dhariyal@huawei.com; xulei139@huawei.com).

Digital Object Identifier 10.1109/TCC.2025.3634737

Unsupervised ML models are predominantly employed to perform anomaly detection as they require no labelled training data [16]. These models tend to focus on detecting deviations from expected patterns, which may not always correspond to meaningful anomalies from a domain perspective, e.g., microservice architectures. Moreover, unsupervised models are often less capable of effectively capturing the subtle, dynamic, and nonlinear relationships between variables in multivariate data. Supervised learning, on the other hand, offers notable advantages when high-quality labelled datasets are available. These methods can learn direct relationships between features and outcomes, enabling better detection of subtle changes in the system that unsupervised methods may miss. Furthermore, supervised approaches provide greater explainability, as they allow practitioners to understand why specific inputs are considered anomalous. However, a key limitation of supervised learning is its reliance on labelled data, which can be costly to obtain, and this is also why unsupervised methods are prevalent in current Multivariate Time Series Anomaly Detection (MTSAD) research.

In our previous work [17], we introduced two high-quality labelled multivariate time series datasets generated from a simulated sock shop microservice application. These datasets are specifically designed to capture realistic anomalies in a microservice-based cloud system. To model the complex relationships present in the datasets, graph-based methods such as Graph Neural Network (GNN) models can offer benefits [18]. While most existing graph-based approaches are unsupervised in the literature [19], [20], [21], the availability of high-quality labelled datasets enables the exploration of supervised methods for MTSAD, offering insight which cannot be easily obtained from the unsupervised approaches. These insights include explainability, allowing cloud providers to effectively interpret results for their customers and improve user experiences [16].

In this work, we extend the Attention-Based Spatial-Temporal Graph Convolutional Network (ASTGCN) model from [22] via an ensemble of three graph-view variants with a learnable attention fusion, and we evaluate its performance and explainability on two proprietary datasets [17] against 12 benchmarks. Our key contributions are:

- We propose a novel ensemble-based spatiotemporal anomaly detection architecture that extends ASTGCN by integrating decisions made from 3 different graph views and a learnable attention-based fusion module. Its effectiveness is evaluated against 12 benchmark models across 2 datasets based on event-wise F1 scores.
- We apply model-specific explanation techniques, TreeSHAP for Random Forest (RF), GNNExplainer for Graph Attention Network (GAT), and attention-based Hyperlink-Induced Topic Search (HITS) analysis for ASTGCN, to identify the most influential features, timestamps, and services in anomalous events, providing actionable explanations in cloud microservice contexts.

The rest of the paper is organised as follows: In Section II, we provide a review of existing anomaly detection solutions and their associated explainability methods. We define the problem statement in Section III. The methodology is detailed in Section IV, followed by the experimental setup in Section V. Next, we present the experimental results of graph methods in Section VI and a comparative study of anomaly detection models in Section VII. We discuss our findings on explainability in Section VIII. Finally, we draw our conclusions in Section IX.

II. LITERATURE REVIEW

In this section, we provide a summary of existing studies on multivariate time series anomaly detection, especially in the context of microservice applications. We then explore graph-based ML approaches and discuss their applications in detail, especially in MTSAD. Finally, we review the tools and methods commonly used to enhance the explainability of ML models, which are essential to interpreting the predictions of complex models in cloud environments. These three aspects, i.e., anomaly detection, graph-based approaches, and model explainability, jointly form the foundation of our work.

A. Multivariate Time Series Anomaly Detection

In multivariate time series analysis, especially in microservice applications, classical ML methods such as k-means clustering, support vector machines, and RF have been widely applied for anomaly detection [13], [14], [15]. While these techniques offer strong explainability, they often fall short when dealing with high-dimensional data and complex temporal dependencies [23]. To overcome these challenges, DL models have emerged as more powerful alternatives, with particular effectiveness in anomaly detection for high-dimensional datasets.

One popular DL approach is the Autoencoder (AE), which detects anomalies by reconstructing input data and calculating reconstruction errors as anomaly scores [24]. Based on this, an Variational Autoencoder (VAE)-based approach was proposed in [25], which captures robust local features in short temporal windows and combines them with Long Short-Term Memory (LSTM) networks for long-term correlations. Another class of methods focuses on prediction-based anomaly detection, where models predict future data based on historical patterns and compare these predictions with actual values to identify deviations. A notable example is the Dynamic Attention Model (DAM) [26], which enhances the prediction power of LSTM models by incorporating an attention mechanism. This mechanism allows the model to selectively focus on both temporal and interdimensional dependencies across multimodal data, improving the accuracy of future value predictions [21]. Additionally, DAM integrates a dynamic threshold selection algorithm based on Peaks Over Threshold (POT) [27], ensuring precise anomaly detection without requiring expert knowledge.

Beyond prediction-based methods, researchers have introduced state-of-the-art models for time series prediction, including LSTM, Multi-Layer Perceptron (MLP), and Transformer architectures [28], [29], [30], [31], [32], [33], [34]. These models excel in capturing complex temporal relationships within multivariate datasets, especially in microservice applications, enhancing their capabilities in anomaly detection [35].

However, most existing works either focus on flat time-series modelling or assume fixed input structures, and few are designed to handle both the dynamic inter-service dependencies and asynchronous behaviours common in cloud-native microservice architectures. This motivates our use of graph-based models for more structured and adaptive representations.

B. Graph Neural Network

Recent advancements in GNN methods have expanded their applicability to tasks involving graph-structured data, such as node classification [36] and edge prediction [37]. Among various GNNs, Graph Convolutional Network (GCN) models have

gained considerable attention due to their ability to leverage spectral domain transformations [38]. GCNs extend the convolution operations to graph structures by incorporating the Laplacian graph matrix, which facilitates efficient message diffusion across nodes. However, traditional GCNs often depend on predefined graph structures, limiting their adaptability in dynamic environments. To address this limit, GAT [39] was proposed based on GNN methods, using attention mechanisms to dynamically establish relationships between nodes by assigning learnable coefficients to each edge. This evolution enables GATs to capture dependencies more flexibly and enhances their applicability to various fields, such as microservice systems [35].

Microservice systems operate in dynamic environments, where anomaly detection requires considering both temporal factors in metrics data and spatial connections between interconnected microservices [40]. Recent models, such as Spatio-Temporal Graph Convolutional Network (STGCN) [41] and ASTGCN [22], have shown success in addressing such complex spatiotemporal tasks, e.g., road traffic prediction. A recent paper [35] applied spatiotemporal GNN-based methods for metric prediction in microservices, which is the first use of such architectures in microservice environments, providing a foundation for anomaly detection in our work. Additionally, recent studies introduced ensemble learning approaches to further enhance the performance of GNNs. For instance, GNN-Ensemble [42] presents a framework combining multiple GNN models trained on randomly selected substructures and features and achieving improved classification performance. Another [43] used a hierarchical ensemble of GNNs to learn relationships between traffic flows at various granularity levels.

Inspired by these approaches, we applied ensemble learning to ASTGCN models in this study, to address MTSAD in microservice environments, capturing diverse spatiotemporal views and improving robustness in different service topologies.

C. Explainability

As discussed above, ML or DL models are efficient and widely adopted for anomaly detection in microservices, but their complexity often makes them characterised as “black boxes” [44], [45], [46] and leads to user concerns related to model reliability and accountability [47]. Thus, explainability is essential for increasing transparency and user confidence, as it provides a clearer understanding of detected anomalies and supports more informed decision-making [48].

Explainability in complex models can be achieved through post-hoc Explainable Artificial Intelligence (XAI) techniques [49], [50], of which some provide insights at both global and local levels, where the former helps reveal a model’s overall decision logic and the latter clarifies individual predictions [45], [48]. Common XAI techniques include Shapley Additive Explanation (SHAP) [51], Local Interpretable Model-agnostic Explanations (LIME) [52], Layer-wise Relevance Propagation (LRP) [53], causal inference [54], accelerated model-agnostic explanations [55], Gradient-weighted Cam [56], and Saliency Maps [57]. Among these, SHAP is widely used because of its cooperative game theory foundation, allowing it to compute Shapley values to assess feature contributions to model predictions. This model-agnostic method provides both global and local explanations [58], so it has been applied in many domains, such as feature selection [59], disease prediction [60], [61], and anomaly detection [50], [62], [63], [64], [65]. For instance, SHAP has been applied in [66] for resource

management in cloud-native microservices, using feature contributions to guide targeted resource reallocation for vertical scaling and enhance microservice performance. Additionally, several variants of SHAP have been introduced, including TreeSHAP, which is specifically designed for tree-based models, and KernelSHAP, a model-agnostic method that, although flexible, can be computationally intensive [67].

In the field of GNNs, several methods have been developed to enhance explainability for graph-related tasks and demonstrate strong performance [68], such as GNNExplainer [69], GNN-LRP [70], and SubgraphX [71]. GNNExplainer is designed to identify compact subgraphs and critical features influencing predictions, which is widely used among these methods, with applications in social network privacy leakage [72] and brain activity analysis [73]. Furthermore, attention weights serve as a means of improving the explainability for attention-based models [58], [74]. Specifically, a deep multi-instance contrastive learning method was introduced in [75] by using dual attention to detect abnormal precursors, clarifying both temporal dynamics and spatial variables. Their work is the first to address the “when” and “where” aspects of anomaly precursor detection, inspiring our work on explainability with spatiotemporal attention-based models like ASTGCN.

However, few existing works integrate multiple explainability methods across different model types or explicitly target cloud microservice environments. Our work fills this gap by combining model-specific explanation techniques, e.g., TreeSHAP, GNNExplainer, attention-based analysis, to offer both global and local interpretability in anomaly detection.

III. PROBLEM STATEMENT

This study focuses on identifying, predicting, and explaining multimodal anomalies in multivariate datasets, so we use a cloud platform scenario as a case study where anomalies can significantly impact system performance and quality. We built a cloud platform and generated data with manually injected multimodal anomalies, following the design in [17]. To address the challenge of detecting and predicting these anomalies, we explored various ML models with different experimental setups and architectures, including supervised, unsupervised, and hybrid algorithms. Finally, we provide a comprehensive analysis of the explainability of the models from both data feature and temporal perspectives. The ML problem is introduced in detail below.

The ML problem is formally defined as follows: we consider the multi-variable time-series anomaly detection task as a classification problem. The input is a sequence of vectors $\{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_T\}$, where each vector $\mathbf{X}_t$ represents d features at time step t . Using a sliding window approach with a window size w , we define a window starting at time t as $\mathbf{W}_t = \{\mathbf{X}_t, \mathbf{X}_{t-1}, \dots, \mathbf{X}_{t-w+1}\}$. Our model, parameterised by θ , predicts a probability $\hat{y}_t = P(y_t | \mathbf{W}_t; \theta)$ for each window $\mathbf{W}_t$. By convention, the labels are typically 0 for normal samples and 1 for anomalies for this binary classification problem, and the appropriate objective function is the binary cross-entropy function, which measures the difference between the predicted probabilities and the actual labels. The objective is to find the optimal set of parameters θ that minimises the binary cross-entropy loss function over all windows. The loss function $\mathcal{L}(\theta)$ is defined as:

$$\mathcal{L}(\theta) = -\frac{\sum_{t=w}^T [y_t \log(\hat{y}_t) + (1 - y_t) \log(1 - \hat{y}_t)]}{T - w + 1} \quad (1)$$

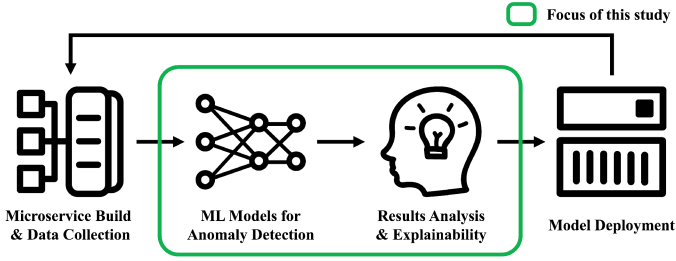


Fig. 1. Overview of workflow architecture.

where $y_t \in \{0, 1\}$ is the actual label for the window starting at position t . The optimisation problem is thus expressed as:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta) \quad (2)$$

In summary, the ML problem in this study involves detecting and classifying anomalies within the current windowed data up to time t . This is formulated as an optimisation task for binary classification aiming to minimise the binary cross-entropy loss function to accurately distinguish between normal samples and anomalies.

IV. METHODOLOGY

The workflow architecture is illustrated in Fig. 1. Building on the two datasets generated from a cloud microservice application as detailed in [17], this study focuses on two primary goals: (1) developing ML models for multivariate anomaly detection in cloud microservices, and (2) enhancing model interpretability through detailed analysis with various explainability techniques. In this section, we focus on the ML model implemented and compared in our work. Specifically, we introduced the benchmark models in terms of supervised, unsupervised, and hybrid methods, and then we detailed the ASTGCN-based ensemble learning framework designed for anomaly detection in a microservice environment.

A. Benchmark Models

Handling non-stationary, multi-dimensional time series data with classical tools is challenging, as discussed in Section II. In contrast, DL models with their non-linear activation functions to extract non-linear relationships in data, are well-suited for time series predictions. To evaluate the performance of our ensemble learning approach, we compare it against 12 DL methods from three categories: supervised algorithms, unsupervised approaches, and hybrid models.

These models were selected based on their effectiveness, frequency, and novelty in recent studies on MTSAD. Specifically, supervised models such as RF, Attention-Based Long Short-Term Memory (ALSTM), Bidirectional Long Short-Term Memory (BiLSTM), and GAT have been widely used in both classic ML and graph-based learning; unsupervised methods like AE and VAE are frequently applied in anomaly detection due to their reconstruction-based strategies; and hybrid models with POT, including recent Transformer-based and MLP-based architectures, such as TimesNet, iTransformer, PatchTST, represent the current state-of-the-art in time series forecasting and are increasingly applied to anomaly detection with dynamic thresholding.

1) Supervised Algorithms:

- *RF [15]*: RF is a traditional ML technique that operates by constructing multiple decision trees during training and outputting the classes or the average prediction of the individual trees.
 - *ALSTM [26]*: LSTM network is a type of recurrent neural network aiming to address the vanishing gradient problem, which employs memory cells and gates to capture temporal features. ALSTM uses attention mechanisms to focus on the most relevant parts of the input.
 - *BiLSTM [28]*: BiLSTM extends standard LSTMs by processing data in both forward and backward directions, capturing dependencies before and after each time step.
 - *GAT [21]*: GAT is a type of GNN model designed to enhance the modelling of relationships using attention mechanisms, which dynamically weighs the importance of neighbouring nodes during aggregation, making the model focus on the most relevant sections of the graph.
- 2) *Unsupervised Algorithms:*
- *AE [24]*: AEs are designed for data compression and unsupervised classification. Their primary objective is to minimise the difference between the original input and its reconstruction, using an encoder-decoder structure.
 - *VAE [25]*: VAEs extend traditional AEs by introducing a probabilistic approach. Specifically, Kullback-Leibler divergence is included in the loss function, which measures how much the learnt latent distribution deviates from the prior distribution.
- 3) *Hybrid Models With POT:*
- *xLSTM [29]*: Extended Long Short-Term Memory (xLSTM) uses mechanisms such as memory mixing and exponential gating, enabling more complex computations and better handling of long-term dependencies.
 - *TimesNet [30]*: TimesNet converts time series data into frequency components. This approach enhances long-term forecasting by analysing the frequency domain, capturing features both within and between these periods.
 - *SOFTS [31]*: Series-Core Fused Time Series Forecaster (SOFTS) is a multivariate prediction model designed based on MLP. It embeds multivariate time series temporally for each channel, captures channel correlations with multiple Star Aggregate Dispatch (STAD) modules, and aggregates series representations into a global core using a centralised STAD structure.
 - *iTransformer [33]*: iTransformer is a variant of the transformer model designed for time series forecasting. This method uses the attention layer to learn multivariate correlations effectively and the feed-forward network to encode representations of the input sequence to capture temporal features and improve prediction accuracy.
 - *DLinear [32]*: DLinear is an MLP-based model used in long-term time series forecasting. DLinear employs a decomposition approach combined with linear layers. It separates the input into trend and seasonal components, processes each with a single-layer linear model, and aggregates the results for prediction.
 - *PatchTST [34]*: Based on transformer models, Patch Time Series Transformer (PatchTST) is designed to enhance forecasting accuracy by using local semantic information and longer look-back windows, improving the effectiveness of transformers in this domain.
- All these hybrid models introduced above predict features at each timestamp, and these predictions are compared with

the observed values to identify anomalies. The anomaly score is computed using the Mean Squared Error (MSE) between the predicted and actual values. To set the anomaly detection threshold, we apply the POT method, which models the upper tail of the anomaly score distribution using a Generalised Pareto Distribution (GPD). The tail is defined as the portion of scores above a high quantile, serving as the initial threshold th . The parameters of the GPD, γ (shape) and β (scale), are estimated using maximum likelihood estimation. The final threshold Th is calculated using the following formula:

$$Th = th + \frac{\beta}{\gamma} \left(\left(\frac{nq}{N_t} \right)^{-\gamma} - 1 \right) \quad (3)$$

where q is the desired probability of observing $X > th$, which corresponds to the acceptable false positive rate. γ and β are the GPD parameters, the shape and scale. n represents the total number of observations, and N_t is the number of peaks, where $X_t > th$. If an anomaly score exceeds the final threshold Th , it is labelled as 1 (anomaly); otherwise, it is labelled as 0 (normal).

B. ASTGCN-Based Ensemble Learning Framework

The ASTGCN is designed to capture both spatial and temporal dependencies within graph-structured time series data. Mathematically, we define the microservice data structure as a weighted graph $G = (V, E, A)$ with N nodes, where V is the set of nodes, E is the set of edges, describing the connections between nodes, and $A \in \mathbb{R}^{N \times N}$ is a predefined adjacency matrix used to model spatial dependencies in G , where A_{ij} indicates the weight of the relationship between nodes v_i and v_j . For each node v_i , the graph convolution operation aggregates information from its neighbours based on A , which is critical in guiding the model to learn from spatially correlated nodes.

The input data is represented by a feature matrix $X \in \mathbb{R}^{N \times F \times T}$, where N is the number of nodes, F is the feature dimension, and T represents the time steps. Temporal dependencies are captured using a sliding window approach, where sequences of previous time steps are used to predict future behaviour. To further enhance this process, a temporal attention mechanism dynamically weighs the importance of different time steps. The temporal attention weights $T_i \in \mathbb{R}^T$ for node v_i help the model focuses on pivotal moments that are more predictive of the target variable, improving its capability to model the temporal evolution of anomalies.

As shown in Fig. 2, each ASTGCN block processes the input data in a specific sequence of layers: (i) the **Spatial Att Layer** first applies spatial attention weights to emphasise important relationships within the adjacency matrix A . The spatial attention weights could be obtained by:

$$S_{ij} = \text{softmax}(\alpha_{ij}) \quad (4)$$

where $S \in \mathbb{R}^{N \times N}$ denotes the spatial attention weights, N is the number of nodes, α_{ij} represents the similarity score between nodes v_i and v_j ; (ii) then spatial convolutional weights are employed in the **Spatial Conv Layer** to aggregate the information from the neighbouring nodes for each node as:

$$z_i = \sum_{j=1}^N S_{ij} \cdot (W_s X_j) \quad (5)$$

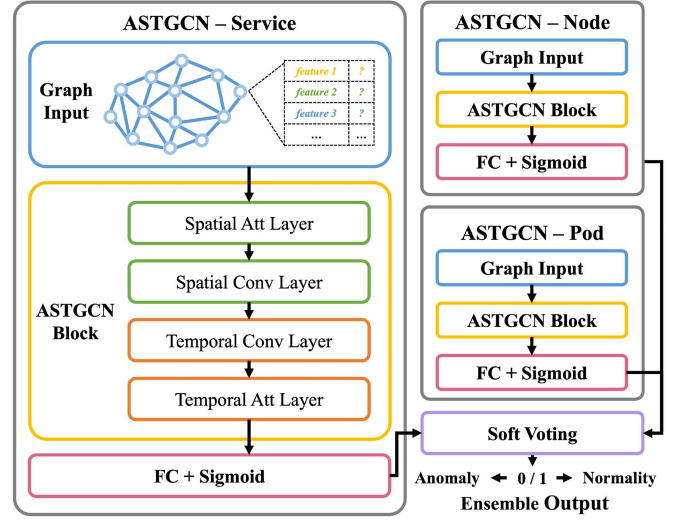


Fig. 2. Structure of the ASTGCN-based ensemble framework.

where $W_s \in \mathbb{R}^{F, F'}$ represents the spatial convolutional weights, transforming the input feature X_j to the new feature space. F and F' stand for the dimensions of the original and new feature spaces, respectively, and z_i is the representation of node v_i after spatial aggregation; (iii) after that, the **Temporal Conv Layer** is applied to capture dependencies across time steps using temporal convolutional weights:

$$z_{i,t} = \sum_{\tau=0}^{k-1} W_t^{(\tau)} \cdot x_{i,t-\tau} + b_t \quad (6)$$

where $W_t^{(\tau)}$ denotes the temporal convolutional weights, controlling the impact of the first k time steps, and b_t is a bias term, ensuring the smoothness of the output; (iv) the **Temporal Att Layer** then assigns different levels of importance to each time step through temporal attention weights:

$$x'_i = \sum_{t=1}^T T_{it} \cdot x_{i,t} \quad (7)$$

where T_{it} denotes the temporal attention weights of node v_i at time step t , and x'_i is the weighted sum of features for node v_i over time, capturing the temporal dependencies most relevant for anomaly detection.

The final ensemble model combines predictions from each ASTGCN block representing different graphs. Using a soft voting mechanism with weighted aggregation, predictions across these distinct views are integrated, achieving a comprehensive perspective of the system.

$$\hat{y} = \sigma \left(\frac{1}{M} \sum_{m=1}^M W_m \hat{y}_m + b \right) \quad (8)$$

where $M = 3$, indicating node, pod, and service graphs, $\hat{y}_m$ ($m \in \{1, 2, 3\}$) represents the output of each graph, W_m is their unique weights, and σ is the sigmoid activation function. This ensemble approach leverages spatial and temporal dependencies from all graphs to improve anomaly detection accuracy across various types of anomalies.

TABLE I
ANOMALY RATIO (%) PER ANOMALY TYPE FOR 70/30 AND 80/20 SPLITS

A.T.	13-05-2024				20-05-2024			
	70/30		80/20		70/30		80/20	
	TR.	TE.	TR.	TE.	TR.	TE.	TR.	TE.
1	2.83	2.51	2.78	2.58	3.36	3.17	3.41	2.88
2	0.92	1.39	1.14	0.74	1.28	1.72	1.22	2.13
3	1.40	1.55	1.45	1.44	1.23	0.60	1.08	0.89
4	1.06	1.88	1.33	1.24	1.57	1.29	1.57	1.14
5	1.90	1.22	1.93	0.74	1.39	1.59	1.34	1.88
6	1.01	0.26	0.92	0.25	0.91	0.93	0.83	1.24
7	0.07	0.10	0.09	0.05	0.07	0.03	0.06	0.05
8	0.09	0.07	0.10	0.00	0.14	0.13	0.17	0.00

V. EXPERIMENTS

In the experimental section, we present a detailed overview of the datasets, the model configurations, the specific topologies implemented for the GAT and ASTGCN models, and the evaluation metrics utilised to assess model performance.

A. Datasets

Our experiments are conducted using the Sock Shop Benchmark Microservice System, deployed on Kubernetes¹ to manage container orchestration. The data generation pipeline has been discussed in the previous work [17]. Specifically, we deployed a microservice application on Kubernetes on the Cloud for managed scaling and container orchestration. The Kubernetes cluster, running on virtual machines, hosts an e-commerce application, Sock Shop,² to simulate a realistic microservice environment, with Istio³ capturing service metrics.

To evaluate the proposed ASTGCN model, two types of anomalies, i.e., overload and fault injections, are introduced. Overload anomalies are simulated using Locust,⁴ which generates dynamic user traffic based on parameters such as user count and spawn rate. Fault injections are introduced via a fault injection simulator and applied at both pod and node levels. Pod-level anomalies include pod deletion, CPU and memory stress, network latency, packet loss, and I/O stress, while node-level anomalies involve CPU and memory stress, network latency, packet loss, I/O stress, and instance reboots. Istio manages traffic flows and captures key metrics, while the cloud monitoring tool gathers fault injection metrics and logs, with Prometheus scraping additional Istio metrics.

This setup generated two datasets for time series anomaly detection, corresponding to the periods 13-05-2024 and 20-05-2024. Each dataset contains 10,080 time points (minutes) and 467 features, which include metrics from nodes, pods, and services. Specifically, the dataset consists of 125 metrics from 5 nodes, 252 metrics from 14 pods, and 90 metrics from 10 services. We employed two data split settings for training and testing: (1) a 70/30 split; and (2) an 80/20 split. Table I presents detailed information about the anomaly types and corresponding anomaly ratios (%) across training and testing datasets for both data splits, where **A.T.**, **TR.** and **TE.** represent anomaly type, train, and test. Specifically, anomaly types 1 to 8 represent “overload”, “cpu-stress”, “io-stress”, “memory-stress”, “network-latency”, “packet-loss”, “pod-deletion” and

TABLE II
OVERVIEW OF DL MODEL HYPERPARAMETERS

Method	T. Epochs	LR	Optimiser	Loss F.
ALSTM / BiLSTM	10	$1 \times e^{-3}$	Adam	BCE
GAT	6	$1 \times e^{-2}$	Adam	BCE
ASTGCN	23	$1 \times e^{-3}$	Adam	BCE
AE / VAE	30	$1 \times e^{-3}$	Adam	MSE
xLSTM	10	$5 \times e^{-4}$	Adam	MSE
TimesNet	10	$1 \times e^{-4}$	Adam	MSE
iTransformer / SOFTS	10	$1 \times e^{-3}$	Adam	MSE
DLinear / PatchTST	20	$1 \times e^{-3}$	Adam	MSE

“node-reboot”, respectively. Importantly, the bolded **0.00** indicates that anomaly type 8 (node reboot) is absent from the test set under the 80/20 split. This is caused by using chronological data split, as this type of anomaly was relatively rare and occurred only in the earlier part of the time series. Consequently, it does not appear in the final 20% used for testing and results in a slight inconsistency in their performance results in Table I.

B. Model Setup

We perform all experiments on an Intel Xeon CPU with 13 GB of RAM and a Tesla T4 GPU with 16 GB of memory. The implementation relies on Python, PyTorch, CUDA, and PyTorch-Geometric, with versions selected according to the model requirements. We summarise the generic hyperparameters for each DL model in Table II where **T. Epochs**, **LR**, and **Loss F.** represent training epochs, learning rate, and loss function, respectively. The remaining specific hyperparameters for each model are detailed in the sections below.

1) *ASTGCN*: In our experiments, we evaluate the performance of the proposed ASTGCN model with two setups: (1) individual graphs; and (2) an ensemble learning method integrating these graphs. Specifically, three graphs were created from the entire dataset, corresponding to the pods (14×18 features), nodes (5×25 features) and services (10×9 features), a total of 467 features. Each graph requires an adjacency matrix which is of size (number of nodes $\times$ number of nodes). Each of the three graphs is independently processed by an ASTGCN block consisting of spatial and temporal attention layers, spatial and temporal convolution layers, and batch normalisation. The attended features from each ASTGCN block are passed through a fully connected layer with a sigmoid activation function, producing a binary output for anomaly detection. After training each ASTGCN model independently, we combine predicted values from all three graphs through soft voting with weighted aggregation.

2) *Supervised Models*: RF, ALSTM, BiLSTM, and GAT models are employed at this stage and introduced below.

RF: The RF model is trained on bootstrap samples with random feature selection to reduce inter-tree correlation. Pre-selection techniques, e.g., recursive feature elimination is applied to the RF model to enhance predictive performance.

ALSTM: The ALSTM model consists of three LSTM layers with batch normalisation to capture temporal dependencies. An attention mechanism follows the LSTM layers to emphasise relevant time steps, and a sigmoid activation function is applied at the output layer for binary classification.

BiLSTM: The BiLSTM model includes three Bidirectional LSTM layers with batch normalisation to capture both past and future dependencies. Followed by the output layer, which uses a sigmoid function to generate binary outputs.

¹ <https://kubernetes.io/docs/home/>

² <https://github.com/ocp-power-demos/sock-shop-demo>

³ <https://istio.io/latest/docs/>

⁴ <https://docs.locust.io/en/stable/>

GAT: The GAT model employs a 3-graph structure for pods, nodes, and services, with predictions made independently on each graph and combined by soft voting. Different from ASTGCN, each graph is trained without temporal patterns. The architecture includes two GATConv layers with multi-headed attention (3 heads) to weigh the importance of neighbouring nodes, followed by batch normalisation and ReLU activation after each layer. The output is flattened and processed through a fully connected layer with a sigmoid function.

3) *Unsupervised Models:* We apply AE and VAE unsupervised models for comparison and introduce them below.

AE: The AE model consists of an encoder and a decoder. The encoder compresses the input data using a linear layer and ReLU activation. The decoder reconstructs the input via linear layers, applying a sigmoid activation to ensure output values between 0 and 1. Anomaly scores are computed using MSE between the original input and its reconstruction, with the threshold optimised to maximise the F1 score.

VAE: The VAE model includes an encoder that maps input data to a latent space via a linear layer, generating the mean and log variance of the latent distribution. A reparametrisation step facilitates sampling from this distribution. The decoder reconstructs the input from the latent representation by multiple linear layers with sigmoid activation. Anomaly scores and thresholds are determined as in the AE model.

4) *Hybrid Models:* For the hybrid models, including iTransformer, TimesNet, SOFTS, DLinear, and PatchTST, we retained their original architectures and modified specific hyperparameters. In particular, we set the feature parameter to “M” for multivariate time series prediction, with encoder, decoder, and output dimensions configured to 467. An early stopping mechanism with a patience of 3 is applied to prevent overfitting. For xLSTM, we used “sLSTM” memory cells followed by “xLSTM” blocks to predict 467 metric values.

All hybrid models are trained with a batch size of 32. To optimise the F1 score, we experimented with various window sizes, starting from 3 to 10 with increments of 1, and 10 to 70 with increments of 10. These ranges were selected to capture both short- and long-term temporal dependencies, reflecting various durations of anomaly events observed in our dataset. The (3) is used to determine the threshold, where th is set as the 95th percentile and we fine-tune the q values between 0.01 and 0.06 in steps of $1 \times e^{-3}$ for optimal performance.

C. Implementation of Graph Topology

We implement two settings for the GAT and ASTGCN models: (1) with topology; and (2) without topology. The topology of the pods deployed in our cluster was obtained from an observability tool within the service mesh, as shown in Fig. 3. This topology serves as the basis for constructing the graph-based models. Using adjacency matrices derived from it, we built a directional graph for pods and a bidirectional graph for services, while applying a fully connected graph for cloud nodes. To assess the impact of topology, we compared these models to configurations with fully connected graphs across nodes, pods, and services.

D. Evaluation Metrics

In general anomaly detection tasks, Precision = $TP/(TP + FP)$, Recall = $TP/(TP + FN)$, and F1 score = $2 \cdot (\text{Precision} \cdot$



Fig. 3. Topology of the sock shop microservice architecture.

Recall)/(Precision + Recall) are widely used as evaluation metrics, where TP, FP, and FN refer to true positive, false positive, and false negative, respectively.

However, in practical applications, maintenance engineers typically prioritise event detection over point-wise metrics, as timely identification of fault events is more important than detecting every anomalous point. To address this, we adopt an event-wise F1 evaluation protocol, a recently proposed metric introduced in [76] and further discussed in recent studies [77], [78], where TP and FP are computed at the event level, while FN remains as the count of false negatives at the point level. Additionally, incorporating the False Alarm Rate (FAR) ensures that precision is not inflated by algorithms that trigger alarms too frequently. Given the context above, the event-wise recall R_e is still calculated by the same formula with event-level TP and FN, but the event-wise precision P_e needs to be computed by:

$$P_e = \frac{TP_e}{TP_e + FP_e} \times (1 - \text{FAR}) \quad (9)$$

where TP_e refers to the number of Ground Truth (GT) events overlapping with detected segments, FP_e is the number of detected segments that do not overlap with any GT event, and FAR is calculated as the ratio of false positives to the total number of normal points. Specifically, “segments” represent contiguous points the algorithm deems anomalous, whereas “events” refer to GT anomalous events. The event-wise F1 score $F1_e$ is computed using P_e and R_e at the event level.

VI. GRAPH-BASED RESULTS

This section analyses the results of graph-based methods, focusing on the role of temporal dependencies in the GAT and ASTGCN models, the impact of varying topologies, and the effectiveness of ensemble learning techniques.

A. Temporal Dependencies in Graph Methods

In this section, we compare the performance of ASTGCN and GAT at different levels of the microservice architecture, specifically the pod and node levels. As shown in Table III, ASTGCN consistently outperformed the GAT model, particularly at the pod level (ASTGCN-P), achieving F1 scores of 0.82 and 0.77 across the two datasets. In contrast, GAT models, which focus on spatial relationships through attention mechanisms, performed optimally at the pod level (GAT-P), with F1 scores of 0.77 and

TABLE III
PERFORMANCE COMPARISON OF GAT AND ASTGCN (70/30 SPLIT)

Method	13-05-2024			20-05-2024			Avg F1
	Prec	Rec	F1	Prec	Rec	F1	
GAT-N	0.50	0.35	0.41	0.45	0.20	0.28	0.35
GAT-P	0.81	0.74	0.77	0.77	0.70	0.73	0.75
GAT-S	0.60	0.61	0.61	0.30	0.66	0.41	0.51
GAT-E	0.85	0.86	0.86	0.73	0.70	0.71	0.79
ASTGCN-N	0.70	0.54	0.61	0.76	0.36	0.49	0.55
ASTGCN-P	0.83	0.82	0.82	0.94	0.65	0.77	0.80
ASTGCN-S	0.48	0.68	0.56	0.55	0.75	0.63	0.60
ASTGCN-E	0.90	0.97	0.94	0.87	0.81	0.84	0.89

TABLE IV
PERFORMANCE COMPARISON OF GAT AND ASTGCN (80/20 SPLIT)

Method	13-05-2024			20-05-2024			Avg F1
	Prec	Rec	F1	Prec	Rec	F1	
GAT-N	1.00	0.24	0.39	0.55	0.18	0.27	0.33
GAT-P	0.96	0.71	0.81	0.70	0.67	0.69	0.76
GAT-S	0.47	0.61	0.53	0.36	0.42	0.39	0.46
GAT-E	0.86	0.78	0.82	0.73	0.67	0.70	0.76
ASTGCN-N	0.60	0.39	0.47	0.83	0.45	0.59	0.53
ASTGCN-P	0.94	0.80	0.87	0.74	0.76	0.75	0.81
ASTGCN-S	0.46	0.80	0.59	0.46	0.73	0.57	0.58
ASTGCN-E	0.98	0.98	0.96	0.93	0.89	0.91	0.94

0.73, slightly lower than those of ASTGCN. At the node level, the ASTGCN variant (ASTGCN-N) also demonstrated strong performance. Meanwhile, GAT's performance at the node level (GAT-N) declined significantly, with F1 scores dropping to 0.35.

Our experiments indicate that incorporating temporal dependencies into spatial graphs significantly improved anomaly detection performance in multivariate time series data, especially in a microservice environment where temporal and spatial interactions are both critical. Through temporal convolutional layers, ASTGCN models can effectively capture sequential patterns within microservice pods and nodes. However, in scenarios requiring the detection of time-dependent anomalies, such as at the node level, GAT falls short in capturing the temporal patterns that ASTGCN achieves, resulting in lower detection accuracy.

B. Ensemble Learning Method

As presented in Table III, the individual graph-based models were evaluated against their ensemble counterparts, specifically examining the ASTGCN and GAT architectures. The ASTGCN-E, which incorporates ensemble learning across node, pod, and service levels with a higher weight on the pod-level graph, achieved the highest average F1 score of 0.89 across both datasets. Its precision and recall are notable, with values of 0.90 and 0.97 for the 13-05-2024 dataset and 0.87 and 0.81 for the 20-05-2024 dataset, respectively. The GAT-E ensemble variant also shows an improvement, achieving an average F1 score of 0.79. Table IV reinforces the advantage of ensemble learning under an 80/20 split. The ASTGCN-E model achieved even higher performance metrics, with a precision of 0.98, recall of 0.98, and an F1 score of 0.96 on the 13-05-2024 dataset. On the 20-05-2024 dataset, it reached a precision of 0.93, a recall of 0.89, and an F1 score of 0.91. The GAT-E model also demonstrated improved performance over its individual-level counterparts, due to an average F1 score of 0.76. Unlike the ASTGCN models, GAT relies primarily on attention mechanisms, which improve spatial

TABLE V
PERFORMANCE OF GAT AND ASTGCN WITHOUT TOPOLOGY

Method	13-05-2024			20-05-2024			Avg F1
	Prec	Rec	F1	Prec	Rec	F1	
70/30 Split							
GAT-FP	0.70	0.69	0.70	0.71	0.64	0.67	0.69
GAT-FS	0.34	0.61	0.43	0.29	0.48	0.37	0.40
GAT-FE	0.63	0.81	0.71	0.66	0.48	0.56	0.63
ASTGCN-FP	0.91	0.81	0.85	0.69	0.53	0.60	0.73
ASTGCN-FS	0.49	0.62	0.55	0.34	0.76	0.47	0.51
ASTGCN-FE	0.85	0.97	0.91	0.87	0.70	0.78	0.85
80/20 Split							
GAT-FP	0.79	0.68	0.73	0.84	0.58	0.69	0.71
GAT-FS	0.51	0.59	0.55	0.46	0.51	0.48	0.52
GAT-FE	0.69	0.85	0.76	0.59	0.78	0.67	0.72
ASTGCN-FP	0.78	0.71	0.74	0.83	0.67	0.74	0.74
ASTGCN-FS	0.62	0.24	0.35	0.49	0.75	0.59	0.47
ASTGCN-FE	0.89	0.80	0.84	0.88	0.78	0.83	0.84

relationships within a level. However, when combined through ensemble learning, GAT's capability at each level contributes to an overall view of the system's operation.

The consistent improvement observed in ensemble models across both split ratios shows the advantages of integrating multi-level information in anomaly detection. In a microservice architecture, this ensemble approach is especially effective. Each level (node, pod, and service) represents a unique aspect of the system, capturing information ranging from individual nodes to more aggregated service-level interactions. Ensemble learning allows the model to account for these distinct perspectives simultaneously, which is helpful in detecting various anomalies that arise due to interactions between services in complex microservice environments.

C. Impact of Graph Topology

As presented in Table V, we summarise the results for the GAT and ASTGCN models utilising fully connected pod and service graphs, representing an unknown topology in the microservice architecture. The fully connected cloud node structure combined with these graphs yields the aggregated outcomes for both models. Across both the 70/30 and 80/20 data split settings, models with topology consistently outperformed their fully connected counterparts. Specifically in topology-based configurations, the directional pod and bidirectional service graphs, e.g., GAT-P and ASTGCN-S, demonstrate significant improvements in F1 scores over fully connected variants, i.e., GAT-FP, GAT-FS, ASTGCN-FP, and ASTGCN-FS. The ensemble methods, i.e., GAT-E and ASTGCN-E, which integrate topological information, achieve the highest average F1 scores, with the latter reaching 0.94 and 0.89 for the 80/20 and 70/30 splits, respectively. In contrast, models incorporating fully connected graphs across all nodes, pods, and services, i.e., GAT-FE and ASTGCN-FE, showed reduced performance, with the latter achieving F1 scores of 0.84 and 0.85 for the respective data splits.

These results suggest that incorporating structured connections based on service and pod interactions enables models to better generalise across diverse anomaly scenarios, whereas the absence of defined topology introduces noise, reducing the model performance in anomaly detection, especially within cloud microservice architectures.

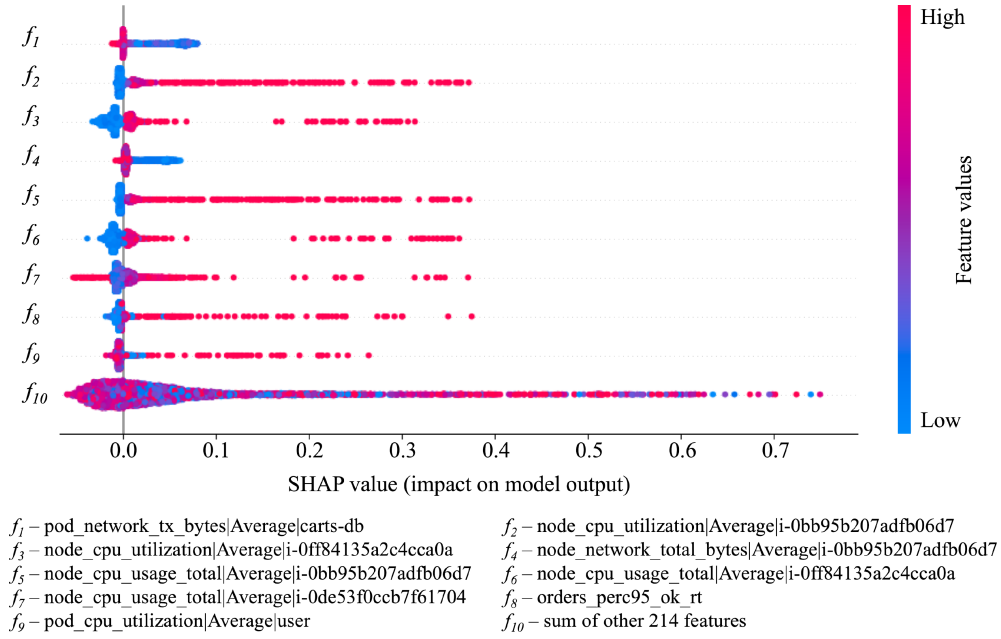


Fig. 4. Feature contribution from TreeSHAP in 13-05-2024 dataset.

TABLE VI
PERFORMANCE COMPARISON OF METHODS ON 13-05-2024 AND 20-05-2024 DATASETS. (70/30 SPLIT)

Method	13-05-2024					20-05-2024					Avg F1
	W. Size	Prec	Rec	Acc	F1	W. Size	Prec	Rec	Acc	F1	
RF	1	0.72	0.94	0.96	0.81	1	0.66	0.88	0.92	0.76	0.79
BiLSTM	27	0.90	0.86	0.96	0.88	12	0.74	0.89	0.96	0.81	0.85
ALSTM	15	0.97	0.60	0.96	0.74	14	0.89	0.84	0.96	0.86	0.80
GAT	1	0.85	0.86	0.96	0.86	1	0.73	0.70	0.91	0.71	0.79
xLSTM	60	0.76	0.54	0.86	0.63	8	0.80	0.64	0.91	0.71	0.67
iTransformer	3	0.67	0.49	0.92	0.56	3	0.67	0.52	0.92	0.59	0.58
PatchTST	9	0.60	0.50	0.92	0.54	10	0.56	0.62	0.91	0.59	0.57
TimesNet	3	0.59	0.50	0.92	0.54	3	0.64	0.54	0.91	0.58	0.56
SOFTS	4	0.57	0.49	0.92	0.53	3	0.59	0.57	0.92	0.58	0.56
DLinear	13	0.60	0.52	0.91	0.56	9	0.64	0.47	0.91	0.54	0.55
AE	1	0.91	0.38	0.94	0.54	1	0.86	0.37	0.93	0.51	0.53
VAE	1	0.89	0.40	0.94	0.55	1	0.84	0.36	0.93	0.49	0.52
ASTGCN-E	9	0.90	0.97	0.97	0.94	9	0.87	0.81	0.95	0.84	0.89

VII. COMPARATIVE STUDY

In this section, we provide a comparative study of model performance, including overall evaluation under different data split ratios, and a discussion of the model performance.

A. Overall Evaluation

Tables VI and VII present a detailed evaluation of various anomaly detection models across the 13-05-2024 and 20-05-2024 datasets, considering event-wise precision, recall, F1 score, and accuracy under two different data split ratios: 70/30 and 80/20. Under the 70/30 split, BiLSTM and our framework, ASTGCN-E, achieved the highest F1 scores of 0.85 and 0.89, respectively, while unsupervised models like AE and VAE showed lower performance, averaging F1 scores of 0.53 and 0.52. Shifting to the 80/20 split resulted in better performance for supervised models, with BiLSTM and ASTGCN-E achieving average F1 scores of 0.93 and 0.94, while unsupervised models kept underperforming, with only marginal improvements in their F1 scores to 0.53 and 0.54.

The performance of benchmark models is summarised below: (1) ALSTM achieved the strongest performance against other supervised models, i.e., RF, BiLSTM, and GAT, although it still underperformed ASTGCN-E; (2) unsupervised models, i.e., AE and VAE, had higher precision but significantly lower recall, resulting in the lowest F1 scores; and (3) the overall performance (F1 score) of hybrid models, e.g., iTransformer, TimesNet, and DLinear, locates between supervised and unsupervised methods.

B. Discussion

The performance evaluation of supervised, hybrid, and unsupervised models provides valuable insights into their capabilities and limitations for multivariate time series anomaly detection. Supervised models, including spatial-temporal graph-based techniques as well as machine learning methods like RF and deep learning approaches such as LSTM variants, demonstrate superior anomaly detection performance by effectively utilising labelled data to capture complex spatial and temporal relationships. However, their reliance on high-quality labelled datasets limits their applicability in real-world scenarios where

TABLE VII
PERFORMANCE COMPARISON OF METHODS ON 13-05-2024 AND 20-05-2024 DATASETS. (80/20 SPLIT)

Method	13-05-2024					20-05-2024					Avg F1
	W. Size	Prec	Rec	Acc	F1	W. Size	Prec	Rec	Acc	F1	
RF	1	0.93	0.92	0.97	0.92	1	0.87	0.73	0.92	0.80	0.86
BiLSTM	27	0.94	0.90	0.98	0.92	12	0.98	0.89	0.97	0.93	0.93
ALSTM	15	0.94	0.93	0.97	0.93	14	0.92	0.89	0.96	0.91	0.92
GAT	1	0.86	0.78	0.96	0.82	1	0.73	0.67	0.94	0.70	0.76
xLSTM	3	0.87	0.51	0.95	0.64	3	0.87	0.62	0.92	0.72	0.68
iTransformer	3	0.54	0.63	0.94	0.58	3	0.71	0.69	0.92	0.70	0.64
TimesNet	7	0.59	0.61	0.94	0.60	3	0.71	0.62	0.91	0.66	0.63
SOFTS	7	0.56	0.59	0.94	0.57	3	0.77	0.60	0.91	0.68	0.63
DLinear	7	0.51	0.63	0.93	0.56	23	0.69	0.73	0.91	0.71	0.64
PatchTST	9	0.47	0.71	0.94	0.57	25	0.68	0.65	0.91	0.67	0.62
AE	1	0.65	0.52	0.93	0.58	1	0.83	0.33	0.93	0.48	0.53
VAE	1	0.63	0.55	0.93	0.59	1	0.84	0.35	0.93	0.49	0.54
ASTGCN-E	15	0.94	0.98	0.98	0.96	15	0.93	0.89	0.96	0.91	0.94

such labels are scarce or costly to obtain. Hybrid models, which integrate unsupervised learning with the threshold selection technique, provide a practical alternative. These models are particularly effective when labelled data is limited, leveraging unsupervised methods to identify potential anomalies and using thresholding techniques to fine-tune detection. Nevertheless, further optimisation of thresholding mechanisms could significantly improve their effectiveness and reliability. By contrast, unsupervised models struggled to match the accuracy of supervised and hybrid approaches, particularly in identifying complex and subtle anomalies in multivariate data. Despite their limitations, they remain valuable in settings where labelled data is unavailable or impractical to obtain.

VIII. EXPLAINABILITY

While Sections VI and VII established the strong detection performance of key models, this section focuses on explaining the reason for their effectiveness. Through model-specific explainability techniques, i.e., SHAP, GNNExplainer, and attention weights, we interpret how models like RF, GAT, and ASTGCN-E made their decisions on the datasets from Section V. This analysis validates our architectural choices and fulfils this study's core objective of providing actionable, interpretable results.

A. Feature Importance

TreeSHAP is used with default setting to analyse feature contributions at a global level for the tree-based model RF. The SHAP value analysis shows that the top 50 features account for approximately 75% of the total feature contributions, highlighting their influence on model predictions. Furthermore, metrics from *front-end*, followed by *order*, *cart*, *user*, and *catalogue*, have a greater predictive impact than those from other sources. Our experimental results revealed several dominant features across both datasets, such as *pod_network_tx_bytes*, *pod_cpu_utilization*, *node_cpu_utilization*, and *rt* (response time), but features like *pod_number_of_running_containers* exhibit greater influence in only one of the datasets, underscoring dataset-specific variations in feature importance.

In Fig. 4, the beeswarm plots of SHAP values illustrate each feature's influence on model predictions, with dots representing individual test instances. Dots on the right (SHAP value > 0) indicate positive contributions to anomaly detection,



Fig. 5. Feature importance from GNNExplainer.

while those on the left (SHAP value < 0) indicate negative contributions. Dot colours range from blue (low feature values) to red (high feature values), reflecting the original feature values. Features are ranked by their mean SHAP values to show their importance. For instance, an increase in *node_cpu_utilization*, *node_cpu_usage_total*, and *services_response_time* correlates with a higher likelihood of anomalies, while *node_network_total_bytes* has a lower probability.

In addition to SHAP, GNNExplainer is applied to analyse feature importance in GAT under a binary classification setting at the graph level. The learning rate and epochs are set to 0.01 and 500, with other parameters at default values. Although GNNExplainer is primarily aimed at generating local explanations, aggregating these local insights still enables us to identify the impactful features for model predictions. According to the feature importance for the pod-level graph visualised in Fig. 5, we observed a strong correlation between the most influential features identified by GNNExplainer and those highlighted by SHAP. However, we also found some differences between them, e.g., *pod_memory_utilisation*, showing a higher impact on GAT model than on RF, which is related to the different characteristics of the models.

Additionally, the spatial attention weights from ASTGCN at the pod level, along with HITS scores [79], reveal the five most influential services: *orders*, *carts*, *user*, *front-end*, and *shipping*, as shown in Fig. 6. These findings closely align with those from the SHAP analysis of the RF model.

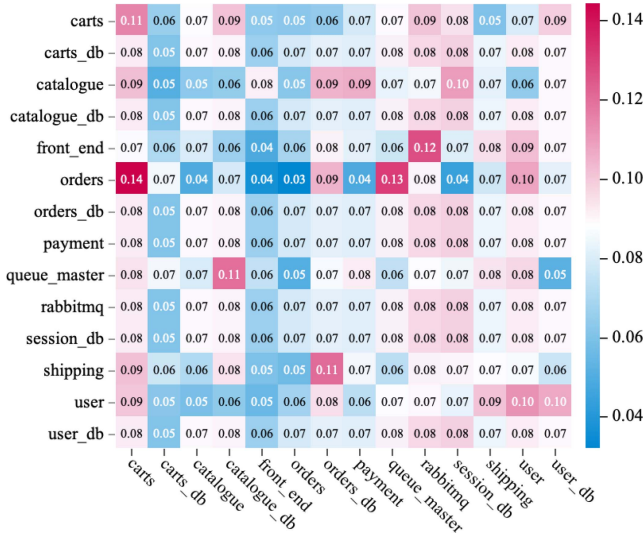


Fig. 6. Pod spatial weights.

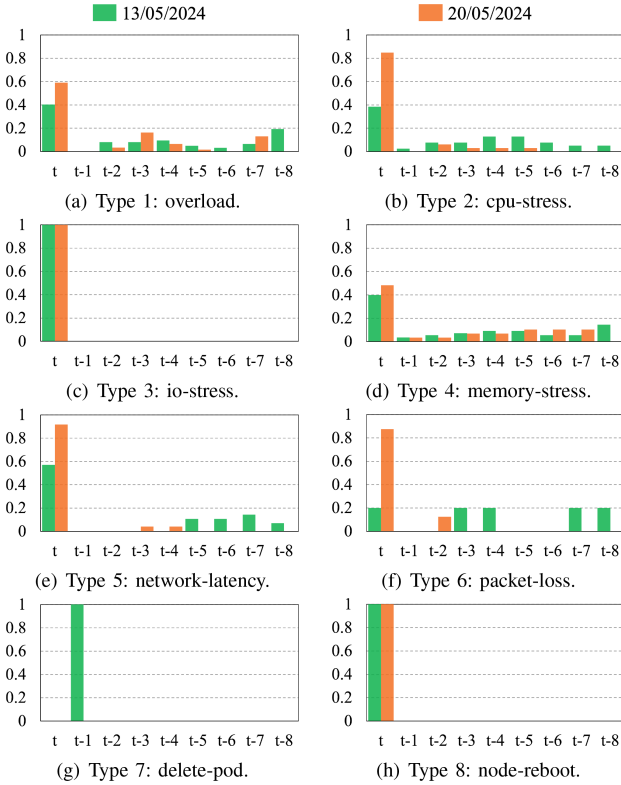


Fig. 7. Temporal dependencies (%) of different anomaly types.

Furthermore, the explainability techniques provide local interpretability by highlighting the specific influence of individual feature values on model predictions, suggesting whether they have a positive or negative effect, and quantifying their contribution to the anomaly score. This capability is particularly valuable for analysing specific instances at particular time steps, enabling a deeper understanding of the model behaviour in targeted cases. For example, when examining anomaly types related to network latency, local explainability reveals that service-level metrics, such as response time, are the major contributors to the anomaly.

B. Time Slice Contribution

The temporal attention weight plots from ASTGCN for correctly predicted anomalies, shown in Fig. 7, reveal distinct timestamp contributions across the two datasets. Key observations from this figure include: (1) in the 13-05-2024 dataset, *Type 7: delete-pod* is uniquely predicted correctly, with timestamp “t-1” as the only contributor; (2) across all anomaly types in both datasets, the current timestamp “t” generally serves as the dominant contributor, with other timestamps having a minimal impact. More specifically, timestamp “t” shows higher temporal attention weights in the 20-05-2024 dataset than 13-05-2024 for most anomaly types; (3) contributions from historical timestamps, ranging from “t-8” to “t-1”, also support the anomaly detection process. For instance, in *Type 6: packet-loss*, timestamp “t” holds a similar level of influence as previous timestamps in dataset 13-05-2024, indicating a balanced impact from both recent and past data.

IX. CONCLUSION & FUTURE WORK

In this paper, we present a novel application of the ASTGCN-based ensemble framework for multivariate time-series anomaly detection in cloud-based microservice architectures. By leveraging spatial and temporal dependencies, our framework consistently achieves superior event-wise F1 scores compared to 12 benchmark models across supervised, unsupervised, and hybrid approaches. Our experiments demonstrate that larger training sets significantly enhance the performance of complex models like BiLSTM and ASTGCN, while graph-based models such as GAT and ASTGCN outperform non-graph models by effectively capturing structured spatial relationships. Temporal modelling further improves detection accuracy, with ASTGCN excelling at capturing sequential patterns. Additionally, ensemble learning across nodes, pods, and services enhances performance by incorporating multi-level system interactions. When topology is unknown, fully connected graphs maintain strong performance, although predefined structures yield higher accuracy.

To enhance model explainability, we apply TreeSHAP for RF, GNNExplainer for GAT, and attention weights with HITS for ASTGCN. TreeSHAP highlights the top 50 features, primarily from the front-end, order, cart, user, and catalogue pods, with key metrics such as *pod_network_tx_bytes*, *pod_cpu_utilization*, and *node_cpu_utilization*. GNNExplainer yields similar results to SHAP but also emphasises *pod_memory_utilization*. Temporal attention highlights recent timestamps as most influential, with earlier ones impacting specific anomalies like overload and packet loss.

Nevertheless, this study has several limitations. First, our evaluation is based on two datasets derived from the same microservice environment, which may limit generalisability to other domains or real-world scenarios. Second, while the ASTGCN-based architecture demonstrates strong performance, its complexity and training cost may hinder deployment in resource-constrained settings. Finally, although we incorporate explainability techniques, fully interpretable graph-based models for MTSAD remain a challenging direction.

As part of our future work, we will focus on several directions aligned with the limitations of this work. Firstly, we aim to generalise our framework across diverse microservice environments and different workloads, possibly through transfer learning or domain adaptation techniques. Secondly, we plan to improve model scalability by reducing computational overhead through model compression or sparsity-aware techniques. Furthermore,

the explainability of graph-based models still needs to be further explored, particularly in extracting human-understandable root causes from detected anomalies. Finally, as our data collection infrastructure evolves, we also plan to explore causal inference methods for explainability, especially those tailored to graph-based architectures. These techniques could help reveal deeper causal dependencies among system components, going beyond attribution-based insights.

REFERENCES

- [1] P. D. Francesco, I. Malavolta, and P. Lago, "Research on architecting microservices: Trends, focus, and potential for industrial adoption," in *Proc. IEEE Int. Conf. Softw. Archit.*, 2017, pp. 21–30.
- [2] B. Żurkowski and K. Zieliński, "Root cause analysis for cloud-native applications," *IEEE Trans. Cloud Comput.*, vol. 12, no. 1, pp. 232–250, Jan. 2024.
- [3] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey," *ACM Comput. Surv.*, vol. 55, no. 3, pp. 1–39, Feb. 2022.
- [4] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng.*, 2020, pp. 48–58.
- [5] Y. Yang and Y. Jiang, "Microservice indicator prediction method based on STE and CNN-BiLSTM," in *Proc. IEEE 9th Int. Conf. Cloud Comput. Intell. Syst.*, 2023, pp. 511–515.
- [6] S. Garg and A. Misra, "Service level agreements for cloud infrastructures," in *Proc. 6th Int. Conf. Comput. Sustain. Glob. Develop.*, 2019, pp. 276–279.
- [7] D. Ma, B. Ding, D. Feng, and H. Liu, "EAD: An efficient anomaly detection algorithm for multivariate time series," in *Proc. IEEE 33rd Int. Conf. Tools Artif. Intell.*, 2021, pp. 609–613.
- [8] W. Sha, Y. Zhu, M. Chen, and T. Huang, "Statistical learning for anomaly detection in cloud server systems: A multi-order markov chain framework," *IEEE Trans. Cloud Comput.*, vol. 6, no. 2, pp. 401–413, Second Quarter 2018.
- [9] B. Agrawal, T. Wiktorowski, and C. Rong, "Adaptive real-time anomaly detection in cloud infrastructures," *Concurrency Computation: Pract. Experience*, vol. 29, no. 24, Aug. 2017, Art. no. e4193.
- [10] X. Zhang et al., "Cross-dataset time series anomaly detection for cloud systems," in *Proc. USENIX Annu. Tech. Conf.*, 2019, pp. 1063–1076.
- [11] M. Ma et al., "Jump-starting multivariate time series anomaly detection for online service systems," in *Proc. USENIX Annu. Tech. Conf.*, 2021, pp. 413–426.
- [12] Y. Chen et al., "Identifying linked incidents in large-scale online service systems," in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Foundations Softw. Eng.*, 2020, pp. 304–314.
- [13] S. Gadal, R. Mokhtar, M. Abdelhaq, R. Alsaqour, E. S. Ali, and R. Saeed, "Machine learning-based anomaly detection using K-mean array and sequential minimal optimization," *Electronics*, vol. 11, no. 14, pp. 2158, Jul. 2022.
- [14] J. Lesouple, C. Baudoin, M. Spigai, and J.-Y. Tournier, "How to introduce expert feedback in one-class support vector machines for anomaly detection?," *Signal Process.*, vol. 188, Nov. 2021, Art. no. 108197.
- [15] I. A. Khan, H. Birkhofer, D. Kunz, D. Lukas, and V. Ploshikhin, "A random forest classifier for anomaly detection in laser-powder bed fusion using optical monitoring," *Materials*, vol. 16, no. 19, Sep. 2023, Art. no. 6470.
- [16] M. A. Belay, S. S. Blakseth, A. Rasheed, and P. S. Rossi, "Unsupervised anomaly detection for IoT-based multivariate time series: Existing solutions, performance analysis and future directions," *Sensors*, vol. 23, no. 5, Mar. 2023, Art. no. 2844.
- [17] M. Allam, N. Boujnah, N. E. O'Connor, and M. Liu, "Synthetic time series for anomaly detection in cloud microservices," in *Machine Learning, Optimization, and Data Science*. Cham, Switzerland: Springer, 2024.
- [18] H. X. Nguyen, S. Zhu, and M. Liu, "A survey on graph neural networks for microservice-based cloud applications," *Sensors*, vol. 22, no. 23, Dec. 2022, Art. no. 9492.
- [19] Y. Sun et al., "Interpretable failure localization for microservice systems based on graph autoencoder," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, Sep. 2024, Art. no. 52.
- [20] Z. He et al., "A spatiotemporal deep learning approach for unsupervised anomaly detection in cloud systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1705–1719, Apr. 2023.
- [21] Z. Liu, X. Huang, J. Zhang, Z. Hao, L. Sun, and H. Peng, "Multivariate time-series anomaly detection based on enhancing graph attention networks with topological analysis," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2024, pp. 1555–1564.
- [22] S. Guo, Y. Lin, N. Feng, C. Song, and H. Wan, "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 922–929.
- [23] C. Wang and G. Liu, "From anomaly detection to classification with graph attention and transformer for multivariate time series," *Adv. Eng. Informat.*, vol. 60, Apr. 2024, Art. no. 102357.
- [24] H. Torabi, S. L. Mirtaheri, and S. Greco, "Practical autoencoder based anomaly detection by using vector reconstruction error," *Cybersecurity*, vol. 6, no. 1, Jan. 2023, Art. no. 1.
- [25] S. Lin, R. Clark, R. Birke, S. Schonborn, N. Trigoni, and S. Roberts, "Anomaly detection for time series using vae-LSTM hybrid model," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2020, pp. 4322–4326.
- [26] Y. Chen, M. Yan, D. Yang, X. Zhang, and Z. Wang, "Deep attentive anomaly detection for microservice systems with multimodal time-series data," in *Proc. 2022 IEEE Int. Conf. Web Serv.*, 2022, pp. 373–378.
- [27] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, "Anomaly detection in streams with extreme value theory," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 1067–1075.
- [28] M. Pirani, P. Thakkar, P. Jivrani, M. H. Bohara, and D. Garg, "A comparative analysis of ARIMA, GRU, LSTM and BiLSTM on financial time series forecasting," in *Proc. 2022 IEEE Int. Conf. Distrib. Comput. Elect. Circuits Electron.*, 2022, pp. 1–6.
- [29] M. Beck et al., "xLSTM: Extended long short-term memory," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 107547–107603, 2024.
- [30] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "Timesnet: Temporal 2D-variation modeling for general time series analysis," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [31] L. Han, X.-Y. Chen, H.-J. Ye, and D.-C. Zhan, "Softs: Efficient multivariate time series forecasting with series-core fusion," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 64145–64175, 2024.
- [32] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 11121–11128.
- [33] Y. Liu et al., "itransformer: Inverted transformers are effective for time series forecasting," 2023, *arXiv:2310.06625*.
- [34] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [35] A. Golovkina, D. Mogilnikov, and V. Ruzhnikov, *Graph Neural Networks for Metrics Prediction in Microservice Architecture*. Cham, Switzerland: Springer, 2024, pp. 343–357.
- [36] S. Xiao, S. Wang, Y. Dai, and W. Guo, "Graph neural networks in node classification: Survey and evaluation," *Mach. Vis. Appl.*, Springer, vol. 33, no. 1, 2022, Art. no. 4.
- [37] M. Chen and Y. Zhang, *Link Prediction Based on Graph Neural Networks*. Red Hook, NY, USA: Curran Associates, Inc., 2018, pp. 5165–5175.
- [38] M. Defferrard et al., "Convolutional neural networks on graphs with fast localized spectral filtering," in *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, Eds. Red Hook, NY, USA: Curran Associates, Inc., 2016.
- [39] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
- [40] L. Chen, "Microservices: Architecting for continuous delivery and DevOps," in *Proc. 2018 IEEE Int. Conf. Softw. Archit.*, 2018, pp. 39–39.
- [41] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. Twenty-Seventh Int. Joint Conf. Artif. Intell.*, 2018, pp. 3634–3640.
- [42] W. Wei, M. Qiao, and D. Jadav, "Gnn-ensemble: Towards random decision graph neural networks," in *Proc. 2023 IEEE Int. Conf. Big Data*, Dec. 2023, pp. 956–965.
- [43] R. Abu Bakar, L. De Marinis, F. Cugini, and F. Paolucci, "FTG-net-E: A hierarchical ensemble graph neural network for DDoS attack detection," *Comput. Netw.*, vol. 250, Aug. 2024, Art. no. 110508.
- [44] E. Dağlarlı, *Explainable Artificial Intelligence (XAI) Approaches and Deep Meta-Learning Models*. London, UK: IntechOpen, Dec. 2020.
- [45] A. Verdone, S. Scardapane, and M. Panella, "Explainable spatio-temporal graph neural networks for multi-site photovoltaic energy production," *Appl. Energy*, vol. 353, Jan. 2024, Art. no. 122151.
- [46] E. S. Ortigossa, T. Gonçalves, and L. G. Nonato, "Explainable artificial intelligence (XAI)—from theory to methods and applications," *IEEE Access*, vol. 12, pp. 80799–80846, 2024.

- [47] D. Branley-Bell, R. Whitworth, and L. Coventry, *User Trust and Understanding of Explainable AI: Exploring Algorithm Visualisations and User Biases*. Berlin, Germany: Springer, 2020, pp. 382–399.
- [48] Z. Li, Y. Zhu, and M. Van Leeuwen, “A survey on explainable anomaly detection,” *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 1, pp. 1–54, Sep. 2023.
- [49] U. Schlegel, H. Arnout, M. El-Assady, D. Oelke, and D. A. Keim, “Towards a rigorous evaluation of XAI methods on time series,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop*, 2019, pp. 4197–4201.
- [50] U. Do, L. Lahesoo, R. M. Carnier, and K. Fukuda, “Evaluation of XAI algorithms in IoT traffic anomaly detection,” in *Proc. 2024 Int. Conf. Artif. Intell. Inf. Commun.*, 2024, pp. 669–674.
- [51] S. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4768–4777.
- [52] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should i trust you? Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1135–1144.
- [53] S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, and W. Samek, “On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation,” *PLoS One*, vol. 10, no. 7, Jul. 2015, Art. no. e0130140.
- [54] D. Janzing, D. Balduzzi, M. Grosse-Wentrup, and B. Schölkopf, “Quantifying causal influences,” *Ann. Statist.*, vol. 41, no. 5, Oct. 2013, pp. 2324–2358.
- [55] D. Dandolo, C. Masiero, M. Carletti, D. Dalle Pezze, and G. A. Susto, “Acme—accelerated model-agnostic explanations: Fast whitening of the machine-learning black box,” *Expert Syst. Appl.*, vol. 214, Mar. 2023, Art. no. 119115.
- [56] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” *Int. J. Comput. Vis.*, vol. 128, no. 2, pp. 336–359, Oct. 2019.
- [57] K. Simonyan, A. Vedaldi, and A. Zisserman, “Deep inside convolutional networks: Visualising image classification models and saliency maps,” in *Workshop Int. Conf. Learn. Representations*, 2014.
- [58] L. Cummins et al., “Explainable predictive maintenance: A survey of current methods, challenges and opportunities,” *IEEE Access*, vol. 12, pp. 57574–57602, 2024.
- [59] K. Roshan and A. Zafar, “Using kernel shap XAI method to optimize the network anomaly detection model,” in *Proc. 9th Int. Conf. Comput. Sustain. Glob. Develop.*, 2022, pp. 74–80.
- [60] S. S. Haneesha Samudrala, J. Thambi, S. R. Vadluri, A. Mahalingam, and P. B. Pati, “Enhancing parkinson’s disease diagnosis using speech analysis: a feature subset selection approach with lime and shap,” in *Proc. 3rd Int. Conf. Innov. Technol.*, 2024, pp. 1–5.
- [61] C.-H. Lin and C.-L. Liu, “Prediction of blood glucose concentration based on optiscanner and XGBoost in ICU,” *IEEE Access*, vol. 11, pp. 116524–116533, 2023.
- [62] R. R. Karn, P. Kudva, H. Huang, S. Suneja, and I. M. Elfadel, “Cryptomining detection in container clouds using system calls and explainable machine learning,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 674–691, Mar. 2021.
- [63] C. Hwang and T. Lee, “E-SFD: Explainable sensor fault detection in the ics anomaly detection system,” *IEEE Access*, vol. 9, pp. 140470–140486, 2021.
- [64] H. Choi, D. Kim, J. Kim, J. Kim, and P. Kang, “Explainable anomaly detection framework for predictive maintenance in manufacturing systems,” *Appl. Soft Comput.*, vol. 125, Aug. 2022, Art. no. 109147.
- [65] N. Aslam et al., “Interpretable machine learning models for malicious domains detection using explainable artificial intelligence (XAI),” *Sustainability*, vol. 14, no. 12, Jun. 2022, Art. no. 7375.
- [66] M. Mekki, B. Brik, A. Ksentini, and C. Verikoukis, “XAI-enabled fine granular vertical resources autoscaler,” in *Proc. IEEE 9th Int. Conf. Netw. Softwarization*, 2023, pp. 161–169.
- [67] H. Manthena, J. C. Kimmel, M. Abdelsalam, and M. Gupta, “Analyzing and explaining black-box models for online malware detection,” *IEEE Access*, vol. 11, pp. 25237–25252, 2023.
- [68] H. Yuan, H. Yu, S. Gui, and S. Ji, “Explainability in graph neural networks: A taxonomic survey,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5782–5799, May 2023.
- [69] Z. Ying, D. Bourgeois, J. You, M. Žitnik, and J. Leskovec, *GNNEXPLAINER: Generating Explanations for Graph Neural Networks*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [70] T. Schnake et al., “Higher-order explanations of graph neural networks via relevant walks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 11, pp. 7581–7596, Nov. 2022.
- [71] H. Yuan, H. Yu, J. Wang, K. Li, and S. Ji, “On explainability of graph neural networks via subgraph explorations,” in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2021, pp. 12241–12252.
- [72] L.-J. Huang and C.-T. Li, “Partial data, potential exposure: Evaluating privacy leakage via gnnexplainer on social networks,” in *Proc. 2024 Int. Conf. Consum. Electron.*, 2024, pp. 355–356.
- [73] M. Zhdanov, S. Steinmann, and N. Hoffmann, “Investigating brain connectivity with graph neural networks and gnnexplainer,” in *Proc. 26th Int. Conf. Pattern Recognit.*, 2022, pp. 5155–5161.
- [74] W. Samek, G. Montavon, S. Lapuschkin, C. J. Anders, and K.-R. Müller, “Explaining deep neural networks and beyond: A review of methods and applications,” *Proc. IEEE*, vol. 109, no. 3, pp. 247–278, Mar. 2021.
- [75] D. Xu et al., “Deep multi-instance contrastive learning with dual attention for anomaly precursor detection,” *Soc. Ind. Appl. Math.*, Jan. 2021, pp. 91–99.
- [76] M. El Amine Sehili and Z. Zhang, “Multivariate time series anomaly detection: Fancy algorithms and flawed evaluation methodology,” in *Performance Evaluation and Benchmarking*, N. Raghunath, P. Meikel, Eds., Cham, Switzerland: Springer, 2024, pp. 1–17.
- [77] H. Si et al., “Timeseriesbench: An industrial-grade benchmark for time series anomaly detection models,” in *Proc. IEEE 35th Int. Symp. Softw. Rel. Eng.*, 2024, pp. 61–72.
- [78] Q. Tang, C. Dai, Y. Wu, and H. Zhou, “Mlp-mixer based masked autoencoders are effective, explainable and robust for time series anomaly detection,” *Proc. VLDB Endowment*, vol. 18, no. 3, pp. 798–811, Nov. 2024.
- [79] J. M. Kleinberg, “Authoritative sources in a hyperlinked environment,” *J. ACM*, vol. 46, no. 5, pp. 604–632, Sep. 1999.



Kevin O'Shea received the MSc degree from the Technological University of the Shannon, Ireland, in 2023, and works as a research assistant with the Insight Research Ireland Centre for Data Analytics. His research interests include graph neural networks, large language models and generative adversarial networks, particularly in the area of music, audio, and cloud computing.



Sen Yan (Graduate Student Member, IEEE) received the BEng (1st Hons) degree in Internet of Things engineering from Shandong University, in 2018, and the MSc (1st Hons) degree in computer science from the Trinity College Dublin, University of Dublin, in 2020. He is currently working toward the PhD degree in intelligent transportation systems with the School of Electronic Engineering, Dublin City University. He is working in the areas of data-driven methods used in intelligent transportation systems.



Ming Yu received the 1st-Class Honours bachelor's degree in data science and the 1st-Class Honours master's degree in computer science from DCU, Ireland. She is a research assistant working with the Insight Research Ireland Centre for Data Analytics, Dublin City University (DCU). Her research focuses on DevOps and machine learning applications in cloud computing. As a certified Cloud Practitioner, she possesses expertise in cloud network troubleshooting, Linux, and Site Reliability Engineering (SRE) operations.



Xianjuan Chen received the master's degree with first-class honours in computing, data analytics from Dublin City University, in 2024. Her research interests include machine learning and data analysis for applications in smart energy and cloud computing. She is dedicated to leveraging data-driven solutions to solve real-world challenges. Throughout her career, she has actively contributed to research projects and technical workshops, gaining hands-on experience in deploying machine learning to address practical problems.



Stefano Mauceri received the PhD degree from University College Dublin (UCD), Dublin, Ireland, in 2021. His PhD thesis explored one-class time series classification. He is currently a senior research engineer with Huawei Ireland Research Center, Dublin. In general, he has contributed to several projects to enable data-driven analysis and decision-making. His current research interest includes enhancing the observability of complex software systems to ensure reliable software infrastructures.



Bhaskar Dhariyal received the PhD degree from University College Dublin, in 2024, where the key focus was study of the scalable methods for multivariate time-series classification method. He is a time-series researcher with Huawei Ireland Research Centre. His daily work focusses on time-series methods on cloud applications.



Lei Xu has served as an AIOps Architect and Analytics lead with the Huawei Ireland Research Center since 2019. His work focuses on researching and applying AI and data analytics solutions to address challenges in system operations, maintenance, and architecture within the cloud domain. Before joining Huawei, he was an experienced and dedicated applied scientist with years of experience in the banking and IT industries, specializing in AI, machine learning, and cloud computing.



Noel E. O'Connor (Member, IEEE) was the academic director of DCU's Research and Enterprise Hub on Information Technology and the Digital Society, with the responsibility of coordinating multidisciplinary ICT-related research across the university. He is currently a full professor with the School of Electronic Engineering, Dublin City University (DCU), Ireland. He is also CEO of the Insight Research Ireland Centre for Data Analytics, Ireland's largest publicly funded research centre. Since 1999, he has published more than 400 peer-reviewed publications, made 11 standard submissions, and filed 7 patents. The focus of his research is on multimedia content analysis using machine learning and AI, for applications in security/safety, autonomous vehicles, medical imaging, ambient assisted living, multimedia content-based retrieval, smart cities, precision agriculture, and environmental monitoring. He is a member of the ACM, an Area Editor of Signal Processing: Image Communication (Elsevier) and an associate editor of the Journal of Image and Video Processing (Springer).

has published more than 60 papers to date including journals "*IEEE Transactions on Smart Grid*", "*IEEE Transactions on Intelligent Transportation Systems*", "*IEEE Transactions on Automation Science and Engineering*", "*IEEE Systems Journal*", "*IEEE Transactions on Transportation Electrification*", "*IEEE Transactions on Artificial Intelligence*", "*Sustainable Cities and Society*", "*Pattern Recognition*", "*Applied Energy*", and "*Automatica*". He acts as an academic editor for PLOS One and an associate editor for the *Journal of Information Systems and Operational Research* (INFOR). His research interests include control, optimisation and machine learning with applications to IoT, cloud computing, electric vehicles, smart grids, smart transportation, and smart cities.



Mingming Liu (Senior Member, IEEE) received the BEng (1st Hons) degree in electronic engineering from the National University of Ireland Maynooth, in 2011, and the PhD degree in control engineering and decision science from the Hamilton Institute, National University of Ireland Maynooth, in 2015. He is an assistant professor with the School of Electronic Engineering, Dublin City University (DCU). He is also affiliated with the Insight Research Ireland Centre for Data Analytics as a Funded Investigator. Prior to DCU, he worked with University College Dublin

and IBM Ireland Lab as an applied Researcher and EU H2020 project lead. He has published more than 60 papers to date including journals "*IEEE Transactions on Smart Grid*", "*IEEE Transactions on Intelligent Transportation Systems*", "*IEEE Transactions on Automation Science and Engineering*", "*IEEE Systems Journal*", "*IEEE Transactions on Transportation Electrification*", "*IEEE Transactions on Artificial Intelligence*", "*Sustainable Cities and Society*", "*Pattern Recognition*", "*Applied Energy*", and "*Automatica*". He acts as an academic editor for PLOS One and an associate editor for the *Journal of Information Systems and Operational Research* (INFOR). His research interests include control, optimisation and machine learning with applications to IoT, cloud computing, electric vehicles, smart grids, smart transportation, and smart cities.